DLoc Lightweight WiFi Localization

Complete Project Guide — environment, GitHub, experiments, and the NILES paper/slides

E-JUST graduation project

October 18, 2026

This is the single reference for the whole project. By the end you will have an **exact copy of the working folder** — the code, the ∼70 GB WILD dataset, the trained model weights, the shared Claude "memory", and all of the paper/thesis/ slide sources for NILES. Everything you need to present and to continue the work.

Follow the steps **in order**. You already have all the access (GitHub write + the HuggingFace org); you only have to pull everything down. Budget ∼30–60 min, mostly the dataset download.

1 Where everything lives

The project is split across a code repo (small, in git) and HuggingFace (the big files). A `git clone` gives you the code and *all the writing/slides*; you pull the dataset and weights from HuggingFace.

- **GitHub** — `Abdulrahman-bits2/DLoc-lightweight-localization`: code, paper, thesis, slides, figures, reports. *You have write access.*
- **HF** `JustAGeek/dloc-wild-fig10b`: the WILD dataset (∼70 GB). Public.
- **HF** `JustAGeek/dloc-code`: trained weights, logs, results. Public.
- **HF** `Graduation-project123/dloc-private`: the Claude project memory. Private — reached through your org membership (`Ahmed-Emad-Haggag`).

2 Prerequisites (install once)

- **git**, **Python 3.9+** (3.10–3.13 fine) with `pip`.
- Your **HuggingFace account** `Ahmed-Emad-Haggag` (already in the org).
- A **GPU + CUDA** *only to train.* To read code, reproduce the published numbers from the downloaded weights, build the paper, or present — no GPU needed.
- To **edit/compile the paper** locally: a LaTeX install (TeX Live / MiKTeX) or just use Overleaf (Section 8). To open the **slides**: Microsoft PowerPoint.

3 Get the full environment & folder — the easy way

Four steps. The repo ships `setup.sh` that does the rest.

Step 1 — log in to HuggingFace (needed for the private memory repo):

```
pip install -U "huggingface_hub[cli]"
hf auth login        # paste a token from huggingface.co/settings/tokens (Read is enough)
hf auth whoami       # should print: Ahmed-Emad-Haggag
```

Step 2 — clone the code:

```
git clone https://github.com/Abdulrahman-bits2/DLoc-lightweight-localization.git
cd DLoc-lightweight-localization
```

Step 3 — run the setup script:

```
bash setup.sh                 # asks before the ~70 GB dataset download
#   bash setup.sh --all -y    # everything, no questions (includes the dataset)
#   bash setup.sh --no-dataset # skip the dataset (code + weights + memory only)
#   bash setup.sh --with-teacher  # also regenerate teacher outputs (needs a GPU)
```

It makes a virtual environment, installs dependencies, and downloads the dataset, weights, and memory, then creates the cross-session symlinks.

Step 4 — check it worked:

```
ls code/data code/weights claude-memory
```

You should see the `.mat` dataset files, 10 `.pth` weights, and 20 memory `.md` files. That is the full folder — same as the original.

4 Get the environment — the manual way (alternative)

Same result, by hand, from the cloned folder:

```
python -m venv .venv
source .venv/bin/activate              # Windows: .venv\Scripts\activate
pip install -r code/requirements.txt

cd code
hf download JustAGeek/dloc-wild-fig10b --repo-type dataset --local-dir data   # ~70 GB
hf download JustAGeek/dloc-code --include "weights/*" --local-dir .           # weights/
cd ..
hf download Graduation-project123/dloc-private --repo-type dataset --local-dir .  # claude-
    memory/

# cross-session symlinks (only needed for the LOSO runs):
cd code/data
ln -s dataset_jacobs_Aug16_1.mat     dataset_train_jacobs_Aug16_1.mat
ln -s dataset_jacobs_Aug16_3.mat     dataset_train_jacobs_Aug16_3.mat
ln -s dataset_jacobs_Aug16_4_ref.mat dataset_train_jacobs_Aug16_4_ref.mat
cd ../..
```

The Mamba model only, if you ever run it, needs two extra CUDA-only packages: `pip install mamba-ssm --no-deps` then `pip install causal-conv1d`.

5 GitHub workflow (how we collaborate on the code)

When you cloned from GitHub, your `origin` remote *is* the GitHub repo, so the normal workflow just works. **Always pull before you start**, and **work on a branch**, not directly on `main`.

```
git pull                                  # get everyone's latest changes first

git checkout -b ahmed/my-change           # start your own branch
#   ... edit files ...
git add -A
git commit -m "short description of what I changed"
git push -u origin ahmed/my-change        # pushes to GitHub (origin = GitHub)
```

```
# then open a Pull Request on GitHub to merge your branch into main
```

Useful:

```
git status                 # what changed
git log --oneline -10      # recent history
git checkout main && git pull   # back to main and update
```

Rules:

- **Treat HuggingFace as download-only** — use `hf download`, never `git push` to a HuggingFace remote. The HF repos hold the only copies of the weights/dataset/memory; a force-push can destroy them.
- **Don't commit big files** (datasets, `.pth`/`.pt`, logs, run folders). They are git-ignored on purpose — they live on HuggingFace.

6 The project memory + your own Claude

`claude-memory/` is the project's full history — every run, data split, result, decision, and the paper/slide status (20 notes, indexed by `MEMORY.md`). To pick up exactly where things are:

1. Open **Claude Code** (your own account) *inside* the project folder.
2. Tell it: *"Read `claude-memory/MEMORY.md` and the files it indexes — that is this project's full context."*
3. Claude now knows the whole project (including all the NILES/paper decisions).

7 Running experiments

Run everything from the `code/` folder with the venv active.

```
cd code
source ../.venv/bin/activate              # Windows: ..\.venv\Scripts\activate

# --- verify the PUBLISHED results WITHOUT a GPU (uses the downloaded weights) ---
python evaluation/benchmark_models.py      # efficiency table (needs no data)
python evaluation/test_robustness.py       # set MODEL_TYPE + WEIGHTS_PATH at top of file
python evaluation/eval_cross_session.py     # set MODEL_TYPE + WEIGHTS_PATH at top of file

# --- retrain from scratch (needs a GPU) ---
python training/precompute_teacher.py       # once, before any DISTILLED run
python training/train_distillation.py        # students: edit STUDENT_TYPE / MODE at top
python training/train_unet_advanced.py        # U-Net:  edit SCHEDULER_TYPE / MODE at top
bash    training/run_crosssession_table1.sh    # LOSO: 3 folds x 3 seeds
```

The weight files to use for each model are listed in `REPRODUCIBILITY.md` (the `MODEL_TYPE`/`WEIGHTS_PATH` table). Every data split and every run is documented in `Experiments_and_Data_Splits.pdf`.

8 The NILES paper & slides (for your presentation)

Everything for the talk is already in the repo (you got it with `git clone`).

The slides. Open in PowerPoint:

```
NilesConf/WiFi_Localization_NILES_v6.pptx
```

This is the final 38-slide deck (backup slides merged in, all numbers checked).

The paper (camera-ready). The compiled PDF is ready to read:

```
NilesConf/camera_ready_latex/main.pdf       # the camera-ready paper (read this)
NilesConf/camera_ready_latex/main.tex       # its LaTeX source (+ references.bib, figures/)
NilesConf/submission/main.tex               # same paper, the submission source
```

To **edit & recompile** the paper, two options:

- **Overleaf (easiest):** new project → *Upload* all files from `NilesConf/camera_ready_latex/` → compile. It is self-contained (includes `IEEEtran.cls`, `references.bib`, `figures/`).
- **Locally** (from inside that folder):

```
cd NilesConf/camera_ready_latex
# with Tectonic (one command, handles the bibliography):
tectonic main.tex
# or with a standard TeX Live / MiKTeX install:
pdflatex main && bibtex main && pdflatex main && pdflatex main
```

Figures & numbers (useful while presenting):

- `Figures/generated/` — all publication figures (CDFs, heatmaps, compression).
- `Experiments_and_Data_Splits.pdf` — every data split and H100 run, precisely.
- `reports/phase2_results.tex` + `reports/CROSSSESSION_FINAL_REPORT.md` — the canonical result tables (medians/P90/P99, LOSO vs DLoc).

Also in the repo: `NilesConf/journal_paper/` (extended journal version, `Journal_paper.pdf`), `thesis/` (full thesis + `Final_Thesis.pdf`), `presentation/` (an HTML defense-deck variant), `guide/` (study guide).

9 What the folder looks like (reference)

After setup, the folder matches the original:

```
DLoc-lightweight-localization/
  setup.sh                          one-shot environment setup
  ONBOARDING.md  REPRODUCIBILITY.md    guides (result -> command map)
  Team_Setup_Guide.pdf              <- this guide
  Experiments_and_Data_Splits.pdf   every data split + H100 run
  code/
    core/ models/ training/ evaluation/ configs/ figures/   source
    data/           WILD dataset .mat files        (from HF, after setup)
    weights/        10 trained .pth checkpoints     (from HF, after setup)
    results_crosssession/   final LOSO result JSONs
    requirements.txt
  claude-memory/                    the Claude project memory (from HF, private)
  data_split_idx/                   the 10 original split-index .mat files
  Figures/        generated/ (figures)  figure_data/ (per-sample arrays)
  reports/                          LaTeX/Markdown result reports
  NilesConf/
    WiFi_Localization_NILES_v6.pptx   <- the NILES slides
    camera_ready_latex/   main.tex  main.pdf  references.bib  figures/   <- the paper
    submission/           main.tex (camera-ready source)
    journal_paper/        extended journal version (Journal_paper.pdf)
    NILES2026-CFP.pdf     the call-for-papers
  thesis/  paper/  guide/  presentation/     writing artifacts
  .venv/                            Python environment (after setup)
```

10 Troubleshooting

`hf: command not found` `pip install -U "huggingface_hub[cli]"` and make sure the venv is active.

Private memory "403"/"not found" `hf auth whoami` must print `Ahmed-Emad-Haggag`, and you must be in the `Graduation-project123` org (you are).

Dataset download interrupted Re-run it — `hf download` resumes. Skip it with `bash setup.sh --no-dataset`.

LOSO error about `dataset_train_jacobs_Aug16_*` The symlinks are missing (Section 5); `setup.sh` makes them. On Windows, enable Developer Mode or use `mklink`.

`git push` goes to the wrong place From a GitHub clone, `origin` is GitHub — that is correct. Never add or push to a HuggingFace git remote.

Windows Activate the venv with `.venv\Scripts\activate`; symlinks need Developer Mode. The shared GPU workstation is Linux, where all commands work as written.

Appendix — command cheat-sheet

```
# ---- one-time full setup ------------------------------------------
pip install -U "huggingface_hub[cli]" && hf auth login
git clone https://github.com/Abdulrahman-bits2/DLoc-lightweight-localization.git
cd DLoc-lightweight-localization && bash setup.sh --all -y

# ---- daily git ----------------------------------------------------
git pull
git checkout -b ahmed/my-change
git add -A && git commit -m "..." && git push -u origin ahmed/my-change

# ---- run (from code/, venv active) --------------------------------
cd code && source ../.venv/bin/activate
python evaluation/benchmark_models.py
python evaluation/eval_cross_session.py        # set MODEL_TYPE + WEIGHTS_PATH

# ---- build the paper ----------------------------------------------
cd NilesConf/camera_ready_latex && tectonic main.tex     # or upload the folder to Overleaf

# ---- open the slides ----------------------------------------------
# NilesConf/WiFi_Localization_NILES_v6.pptx   (PowerPoint)
```

Questions? Ping the team, or open the project in Claude Code and point it at `claude-memory/MEMORY.md`. Good luck with the NILES talk!